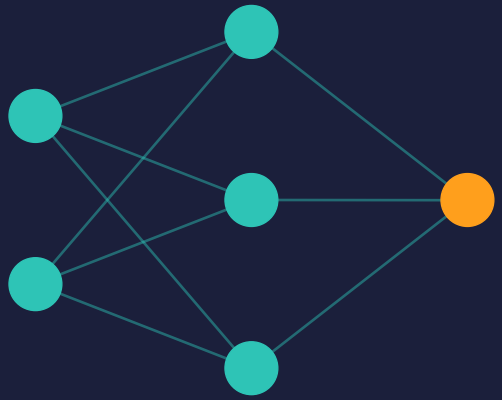
دوره یادگیری عمیق و شبکه‌های عصبی



جلسه ۲

پس‌انتشار خطا و گرادیان کاهشی از صفر

تابع هزینه، گراف محاسباتی، قاعده زنجیره‌ای و آموزش MLP با NumPy

کیوان محمودی

در این جلسه

۱ آموزش = کمینه‌سازی تابع هزینه

۲ توابع هزینه: MSE و Cross-Entropy

۳ گرادیان کاهشی و نرخ یادگیری

۴ گراف محاسباتی و قاعده زنجیره‌ای

۵ ساخت موتور Autograd کوچک

۶ Backpropagation ماتریسی برای MLP

۷ Gradient Checking و Mini-batch SGD

۸ محوشدگی گرادیان (Vanishing Gradient)

سه جزء آموزش یک شبکه عصبی

۱ Loss Function

خطای مدل را به یک عدد تبدیل می‌کند؛ هدف کمینه کردن آن است.

۲ Gradient

جهت بیشترین افزایش هزینه نسبت به هر پارامتر؛ با Backpropagation محاسبه می‌شود.

۳ Optimizer

با استفاده از گرادیان پارامترها را به‌روز می‌کند؛ ساده‌ترینش Gradient Descent است.

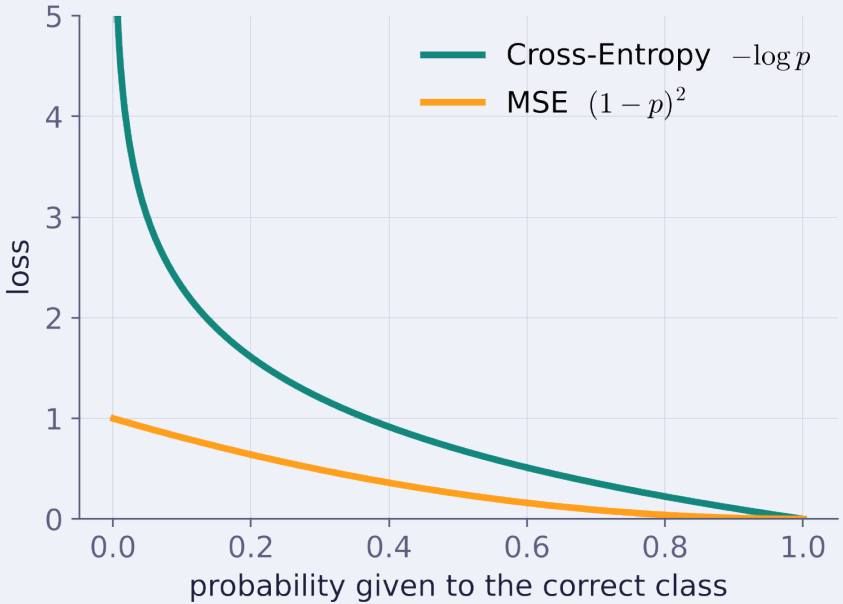
توابع هزینه: MSE و Cross-Entropy

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{i=1}^{N}\log \hat{y}_{i,c_i}$$

- MSE برای رگرسیون؛ Cross-Entropy برای طبقه‌بندی.
- در Cross-Entropy فقط احتمال کلاس درست (c_i) اهمیت دارد.
- کمینه‌کردن Cross-Entropy معادل Maximum Likelihood است.

چرا Cross-Entropy برای طبقه‌بندی؟



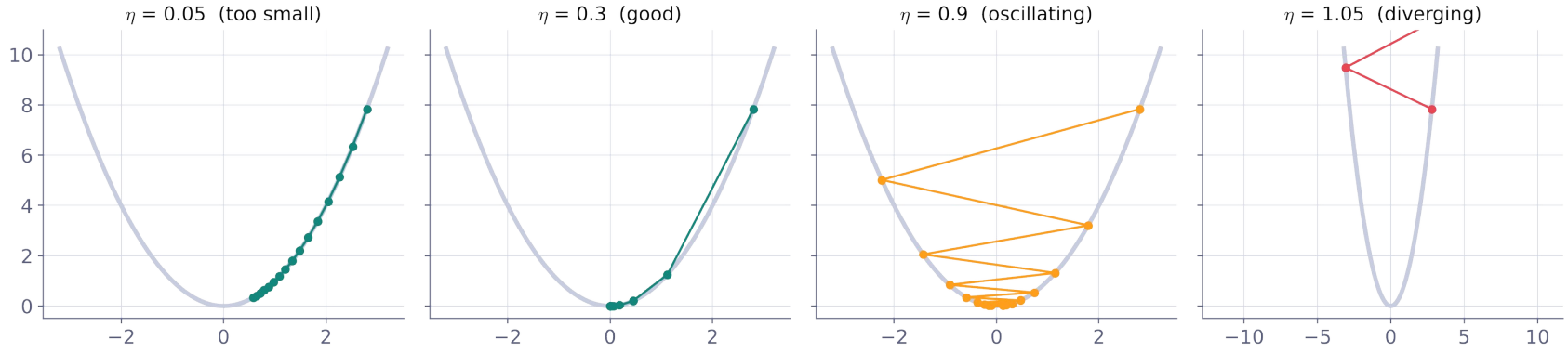
- وقتی مدل با اطمینان اشتباه می‌کند (p → 0)، جریمه Cross-Entropy به بی‌نهایت میل می‌کند.
- جریمه MSE حداکثر ۱ است و گرادیان ضعیفی می‌دهد.
- ترکیب Softmax + Cross-Entropy گرادیان بسیار ساده‌ای دارد.

گرادیان کاهشی (Gradient Descent)

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$$

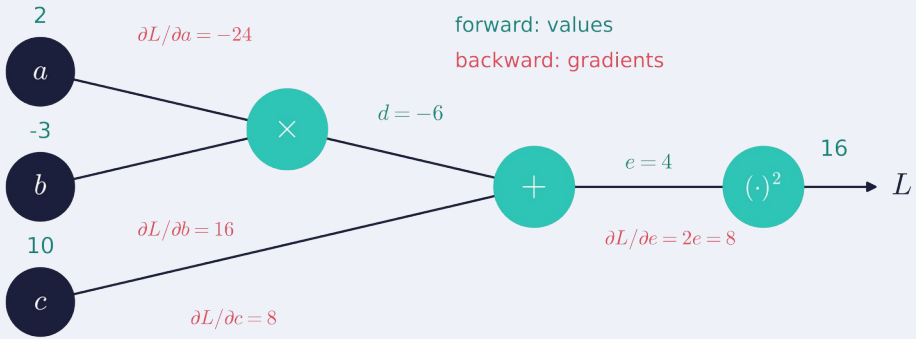
- گرادیان جهت بیشترین افزایش است؛ پس خلاف آن حرکت می‌کنیم.
- η نرخ یادگیری (Learning Rate) و مهم‌ترین ابرپارامتر آموزش است.
- تابع هزینه شبکه‌های عصبی محدب نیست، اما همین ایده ساده در عمل بسیار خوب کار می‌کند.

اثر نرخ یادگیری



کوچک: همگرایی کند | مناسب: سریع | بزرگ: نوسان | خیلی بزرگ: واگرایی

گراف محاسباتی (Computational Graph)



- هر محاسبه = گرافی از عملیات ساده.
- Forward: مقادیر از ورودی به خروجی.
- Backward: گرادیان‌ها از خروجی به ورودی.
- مثال: $L = (a \cdot b + c)^2$

قاعده زنجیره‌ای: قلب Backpropagation

$$\frac{\partial L}{\partial x} = \underbrace{\frac{\partial L}{\partial z}}_{\text{upstream}} \cdot \underbrace{\frac{\partial z}{\partial x}}_{\text{local}}$$

- هر گره فقط مشتق محلی (Local Gradient) خودش را می‌داند.
- گرادیان بالادستی در مشتق محلی ضرب و به پایین فرستاده می‌شود.
- اگر گره‌ای در چند مسیر استفاده شود، گرادیان‌های مسیرها جمع می‌شوند.

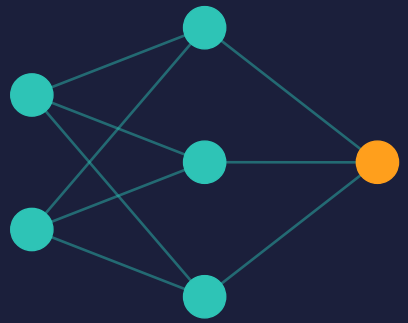
یک موتور Autograd کوچک

```
class Value:
    def __init__(self, data, parents=()):
        self.data, self.grad = data, 0.0
        self._parents = parents
        self._backward = lambda: None

    def __mul__(self, other):
        out = Value(self.data * other.data, (self, other))
        def _backward():
            self.grad  += other.data * out.grad
            other.grad += self.data  * out.grad
        out._backward = _backward
        return out

    def backward(self):
        # visit nodes in reverse topological order
        self.grad = 1.0
        for v in reversed(topological_order(self)):
            v._backward()
```

- هر عملیات، تابع backward خودش را می‌سازد.
- گرادیان‌ها با += انباشته می‌شوند.
- به همین دلیل در PyTorch قبل از هر backward باید zero_grad کنیم.



از اسکالر به ماتریس

Backpropagation برای MLP

محاسبات واقعی روی ماتریس‌ها و دسته‌ای از نمونه‌ها انجام می‌شود

گرادیان‌های یک شبکه دولایه

$$\frac{\partial \mathcal{L}}{\partial Z_2} = \frac{1}{N}(P - Y)$$

$$\frac{\partial \mathcal{L}}{\partial W_2} = H^\top \frac{\partial \mathcal{L}}{\partial Z_2} \qquad \frac{\partial \mathcal{L}}{\partial H} = \frac{\partial \mathcal{L}}{\partial Z_2} W_2^\top$$

$$\frac{\partial \mathcal{L}}{\partial Z_1} = \frac{\partial \mathcal{L}}{\partial H} \odot \mathbb{1}[Z_1 > 0] \qquad \frac{\partial \mathcal{L}}{\partial W_1} = X^\top \frac{\partial \mathcal{L}}{\partial Z_1}$$

- P خروجی Softmax و Y ماتریس One-hot برچسب‌ها است.
- قاعده طلایی: ابعاد گرادیان هر پارامتر = ابعاد خود پارامتر.

Gradient Checking: اعتماد کن، ولی بررسی کن

$$\frac{\partial \mathcal{L}}{\partial \theta} \approx \frac{\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon)}{2\epsilon}$$

- گرادیان تحلیلی را با تقریب عددی تفاضل مرکزی مقایسه می‌کنیم.
- خطای نسبی کمتر از حدود 10^{-6} یعنی پیاده‌سازی درست است.
- فقط برای اشکال‌زدایی؛ برای آموزش بسیار کند است.

Stochastic، Batch یا Mini-batch؟

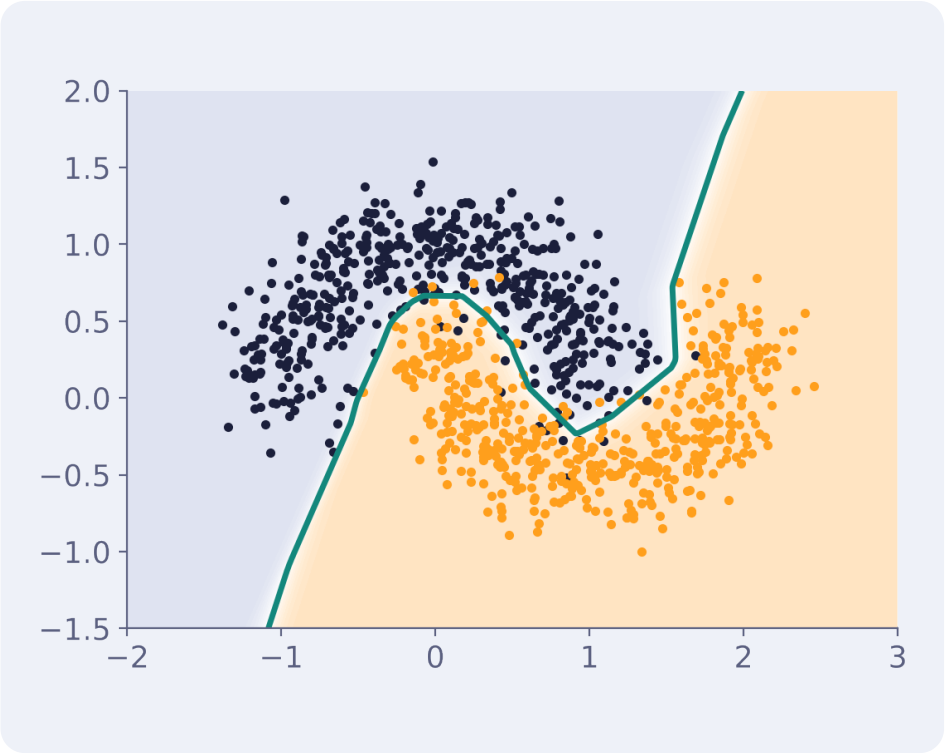
Batch GD و SGD

- Batch: کل داده در هر گام؛ دقیق ولی کند و پرحافظه.
- Stochastic: یک نمونه در هر گام؛ سریع ولی بسیار پرنویز.

Mini-batch GD (استاندارد عملی)

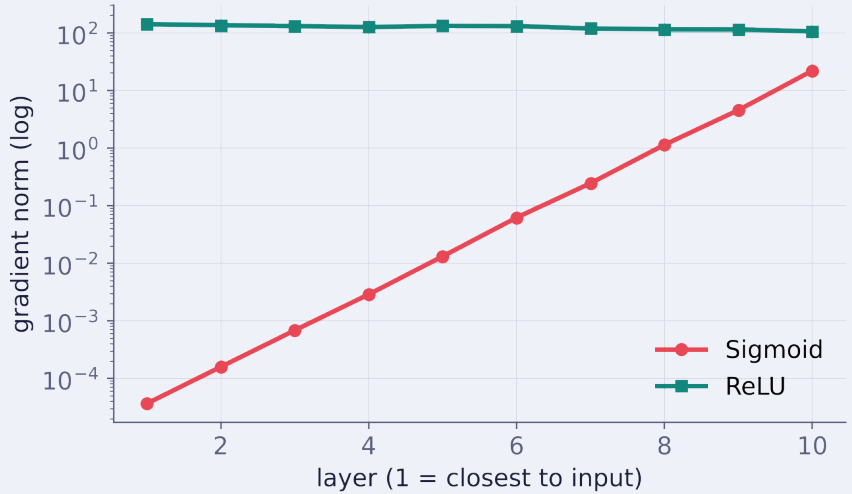
- دسته‌های ۳۲ تا ۲۵۶ نمونه‌ای.
- تعادل بین دقت گرادیان و سرعت.
- مناسب پردازش موازی روی GPU.
- حلقه: Forward → Loss → Backward → Update

نتیجه: MLP دولایه با NumPy روی make_moons



- ۳۲ نورون پنهان با ReLU و خروجی Softmax.
- Mini-batch SGD با اندازه دسته ۶۴ و نرخ یادگیری ۰/۵
- دقت حدود ۹۷٪ روی داده آزمون.
- مرز تصمیم کاملاً غیرخطی است.

محوشدگی گرادیان (Vanishing Gradient)



- گرادیان هر لایه حاصل‌ضرب مشتق‌های لایه‌های بالاتر است.
- بیشینه مشتق Sigmoid برابر ۰/۲۵ است؛ در ۱۰ لایه گرادیان چند میلیون برابر کوچک می‌شود.
- با ReLU و مقداردهی اولیه مناسب، گرادیان تقریباً ثابت می‌ماند.
- راه‌حل‌های بعدی: Initialization، BatchNorm و ResNet.

جمع‌بندی

- Loss خطا را به یک عدد تبدیل می‌کند: MSE برای رگرسیون، Cross-Entropy برای طبقه‌بندی.
- Gradient Descent خلاف جهت گرادیان حرکت می‌کند؛ η مهم‌ترین ابرپارامتر است.
- Backpropagation = قاعده زنجیره‌ای روی گراف محاسباتی.
- Softmax + Cross-Entropy گرادیان ساده P - Y را می‌دهد.
- Mini-batch SGD استاندارد عملی است.
- Sigmoid در شبکه عمیق باعث Vanishing Gradient می‌شود.

تمرین

۱ عملیات tanh و exp را به کلاس Value اضافه کنید و گرادیان را بررسی کنید.

۲ در TwoLayerNet به جای ReLU از Tanh استفاده کنید و با Gradient Checking بررسی کنید.

۳ آموزش را با batch_size برابر ۱، ۶۴ و کل داده مقایسه کنید.

۴ یک لایه پنهان دوم اضافه کنید و فرمول‌های Backward آن را بنویسید.